

Monte Carlo Tree Search pour l’optimisation de l’ordre de jointures : le cas des arbres bushy

Tamazouzt Ait Eldjoudi and Sepanta Farzollahi Zanjani

M2 IASD

14 juillet 2026

Résumé

L’ordre dans lequel un optimiseur de requêtes relationnel effectue les jointures d’une requête multi-tables a un impact déterminant sur le coût d’exécution, et le problème de trouver l’ordre optimal est NP-difficile dès que l’on autorise des plans *bushy* (arbres de jointure non restreints à une chaîne linéaire), ce qui est le cas des optimiseurs réels de type System R/Selinger. Ce travail applique *Monte Carlo Tree Search* (MCTS) à ce problème et le compare à une heuristique gloutonne agglomérative ainsi qu’à une programmation dynamique exacte en $O(3^n)$ servant de vérité terrain sur les petites instances. Sur 60 instances synthétiques ($n = 4$ à 14 tables), Greedy est en moyenne géométrique $166.9\times$ plus coûteux que l’optimum (IC bootstrap 95 % : $[58.9, 469.2]$), contre $1.08\times$ pour MCTS (IC 95 % : $[1.02, 1.16]$), un écart significatif (test de permutation apparié, $p < 5 \times 10^{-5}$). À grande échelle ($n = 16$ à 40, hors de portée du DP exact), MCTS reste plusieurs ordres de grandeur moins coûteux que Greedy. Nous allons cependant au-delà du résultat positif attendu : une expérience à budget de calcul équitable montre que l’avantage de MCTS s’amenuise, voire s’inverse, face à un simple « Greedy randomisé avec redémarrages » lorsque n atteint 25 à budget fixe, et une ablation sur le biais glouton du rollout montre un effet plus modeste que prévu. Ces résultats nuancés sont discutés comme des limites structurelles du budget de calcul plutôt que dissimulés, dans l’esprit d’une évaluation empirique rigoureuse. Enfin, un benchmark complémentaire (Progressive Widening, Sequential Halving appliqué aux arbres, Nested Monte Carlo Search) montre que ce déficit à $n = 25$ n’est pas propre à UCB1 – une autre méthode arborescente à budget fixe (SHOT) s’effondre bien plus sévèrement – et que Progressive Widening n’apporte qu’un remède partiel, spécifique au régime où le problème a été identifié.

Table des matières

1	Introduction	2
2	Formulation du problème	3
2.1	Modèle de graphe de jointure	3
2.2	Modèle de coût	3
2.3	Représentation d’état et actions	3
3	Méthodes	3
3.1	DP exact bushy ($O(3^n)$)	4
3.2	Greedy agglomératif	4
3.3	MCTS	5
3.4	Baselines à budget de calcul équitable	6
4	Protocole expérimental	7

5 Résultats	7
5.1 Petite échelle : comparaison à l'optimum exact	7
5.2 Grande échelle	9
5.3 Convergence	10
5.4 Budget de calcul équitable : d'où vient l'avantage de MCTS ?	10
5.5 Ablation du biais du rollout (p_{greedy})	11
5.6 Étude de cas qualitative	12
6 Benchmark MCTS : trois méthodes complémentaires vues en cours	12
6.1 Rappel : ce à quoi ces méthodes sont comparées	13
6.2 Progressive Widening : cibler directement l'échec à $n = 25$	13
6.3 Sequential Halving appliqué aux arbres (SHOT)	14
6.4 Nested Monte Carlo Search (NMCS)	15
6.5 Synthèse du benchmark	16
7 Discussion et limites	16
8 Conclusion	17

1 Introduction

Face à une requête SQL joignant n tables, un optimiseur de requêtes doit choisir un *plan de jointure* : un ordre (et une forme d'arbre) selon lequel les jointures par paires seront exécutées. Le nombre de plans possibles croît de façon super-exponentielle avec n , et des plans différents peuvent avoir des coûts d'exécution séparés de plusieurs ordres de grandeur, car la taille des résultats intermédiaires dépend fortement de l'ordre choisi. Ce problème d'*ordre de jointure* (*join-order optimization*) est un des problèmes centraux de l'optimisation de requêtes depuis les travaux fondateurs de Selinger et al. [1] sur System R.

Deux familles de plans sont classiquement distinguées :

- les plans **left-deep** (ou en chaîne), où chaque jointure ajoute une nouvelle table à un unique résultat intermédiaire qui grossit progressivement ;
- les plans **bushy**, où deux résultats intermédiaires quelconques peuvent être fusionnés, formant un arbre binaire non restreint.

Une première version de ce projet portait sur les plans left-deep avec un coût défini comme la somme des tailles intermédiaires. Ce cas particulier s'est révélé être un problème **polynomial** connu (algorithme IKKBZ, Ibaraki et Kameda [2], Krishnamurthy, Borat et Zaniolo [3]) pour les graphes de jointure en arbre : Greedy y est alors déjà quasi-optimal par construction, ce qui ne laissait aucune marge pour démontrer un quelconque avantage de MCTS. Nous nous concentrons donc dans ce rapport exclusivement sur le cas **bushy**, qui restaure la difficulté combinatoire réelle exploitée par les optimiseurs de production, et qui est NP-difficile en général.

Pourquoi MCTS ? Le choix d'un plan de jointure bushy peut se formuler comme une séquence de décisions déterministes : partant de n groupes singletons, on fusionne à chaque étape une paire de groupes connectés jusqu'à n'en obtenir plus qu'un seul. Cette formulation correspond exactement au cadre du *single-player MCTS* [6], où l'arbre de recherche porte sur les séquences d'actions d'un unique agent face à un problème d'optimisation combinatoire déterministe, plutôt que sur un jeu adversarial à deux joueurs comme dans l'usage le plus connu de MCTS (AlphaGo). C'est aussi un sujet de recherche actif en bases de données : les optimiseurs appris récents (Neo [8]) et le *Join Order Benchmark* [7] illustrent l'intérêt continu porté à des méthodes de recherche plus informées que les heuristiques gloutonnes classiques.

Contributions. Ce rapport ne se limite pas à démontrer que « MCTS bat Greedy ». Nous :

1. comparons MCTS à un DP exact (vérité terrain) sur des petites instances, avec intervalles de confiance bootstrap et test de significativité ;
2. isolons, via des baselines à budget de calcul équitable, si l’avantage de MCTS vient réellement de la recherche arborescente guidée par UCB1 ou simplement d’un budget de calcul supérieur à Greedy ;
3. quantifions par ablation l’effet du biais heuristique du rollout ;
4. et documentons un cas où MCTS, à budget fixe, cesse d’être la meilleure méthode – un résultat qui nuance la conclusion générale plutôt que de la contredire ;
5. enfin, nous élargissons la comparaison à trois méthodes supplémentaires vues en cours (Progressive Widening, Sequential Halving appliqué aux arbres, Nested Monte Carlo Search – Section 6) pour déterminer si ce dernier résultat est une particularité de UCB1 ou une limite plus générale.

2 Formulation du problème

2.1 Modèle de graphe de jointure

Chaque instance synthétique est un `JoinGraph` de n tables. Chaque table i a une cardinalité $\text{card}(i)$ tirée log-uniformément dans $[1\,000, 2\,000\,000]$ (les tailles de tables réelles s’étendent sur plusieurs ordres de grandeur). Le graphe de jointure est construit en générant d’abord un arbre couvrant aléatoire (garantissant la connexité), puis en ajoutant des arêtes supplémentaires avec probabilité $p_{\text{edge}} = 0.4$ afin d’obtenir des graphes plus denses et donc plus propices à révéler les limites d’une heuristique myope. Chaque arête (u, v) porte une sélectivité tirée uniformément dans $[0.0005, 0.9]$.

2.2 Modèle de coût

Le coût d’une jointure de type hash-join entre deux groupes déjà construits L et R est :

$$\text{cost}(L, R) = \text{size}(L) + \text{size}(R) + \text{size}(L \cup R)$$

où $\text{size}(S) = \prod_{t \in S} \text{card}(t) \times \prod_{e \subseteq S} \text{sel}(e)$ est la cardinalité estimée du résultat joignant les tables de S . Le terme de matérialisation $\text{size}(L \cup R)$ est **omis** lorsque $L \cup R$ correspond à la totalité des n tables : ce terme final est identique pour tout plan valide (c’est la taille de la réponse à la requête) et domine typiquement le coût total de façon écrasante, noyant toute différence réelle entre algorithmes s’il est inclus. L’exclure isole précisément la partie du coût qui reflète la qualité de l’optimisation.

2.3 Représentation d’état et actions

L’état est une *forêt* : un ensemble de groupes de tables déjà jointes, représentée comme un ensemble de `frozenset`. L’état initial contient n singletons. Une action *merge* fusionne deux groupes connectés dans le graphe de jointure (au moins une arête entre les deux groupes) en un seul. Le processus se termine quand il ne reste qu’un seul groupe – le plan de jointure complet. Le coût total du plan est la somme des scans de base plus le coût de chaque fusion effectuée.

3 Méthodes

Les trois méthodes ci-dessous opèrent toutes sur le même espace d’états et le même ensemble d’actions défini en Section 2.3 : un état est une forêt $F = \{G_1, \dots, G_k\}$ de groupes de tables

disjoints (chaque G_i étant un sous-ensemble non vide de $\{1, \dots, n\}$), et une action est un choix de paire (G_i, G_j) connectée dans le graphe de jointure, qui transforme F en $F' = (F \setminus \{G_i, G_j\}) \cup \{G_i \cup G_j\}$. L'état terminal est atteint après exactement $n - 1$ fusions, lorsque $F = \{\{1, \dots, n\}\}$. Ce qui distingue les trois méthodes, c'est uniquement *comment elles choisissent, à chaque étape, la paire à fusionner* – de façon exhaustive et optimale (DP), myope (Greedy), ou guidée par une recherche arborescente stochastique (MCTS).

3.1 DP exact bushy ($O(3^n)$)

Principe. Le DP bushy est une généralisation directe de l'algorithme de Selinger [1] au cas non restreint au left-deep. Il repose sur le *principe d'optimalité de Bellman* : le coût optimal pour construire un groupe de tables $S \subseteq \{1, \dots, n\}$ ne dépend que du coût optimal de construction de ses deux sous-parties, quelle que soit la façon dont on est arrivé à cette partition. On peut donc calculer le coût optimal de tous les sous-ensembles par taille croissante.

Formalisation. Pour tout sous-ensemble de tables $S \neq \emptyset$ (représenté par un masque de bits), on définit récursivement le coût optimal $C^*(S)$ pour joindre exactement les tables de S :

$$C^*(S) = \begin{cases} \text{card}(i) & \text{si } S = \{i\} \text{ (une seule table)} \\ \min_{\substack{\emptyset \subsetneq T \subsetneq S \\ T \text{ et } S \setminus T \text{ connectés}}} \left(C^*(T) + C^*(S \setminus T) + \text{cost}(T, S \setminus T) \right) & \text{sinon.} \end{cases}$$

La contrainte « T et $S \setminus T$ connectés » signifie qu'il existe au moins une arête du graphe de jointure entre une table de T et une table de $S \setminus T$ – une jointure entre deux groupes sans arête commune produirait un produit cartésien, exclu ici. Le coût du plan optimal global est $C^*(\{1, \dots, n\})$, obtenu pour $S =$ ensemble complet des tables. On mémorise en même temps, pour chaque S , la partition $T^*(S)$ qui atteint ce minimum (`dp_split` dans le code), ce qui permet de reconstruire l'arbre de jointure optimal *a posteriori* par récursion inverse, et pas seulement son coût.

Complexité. Chaque sous-ensemble S de taille k possède $2^k - 2$ partitions non triviales ($T, S \setminus T$) à essayer (chacune étant comptée deux fois, T et $S \setminus T$ étant interchangeables, ce que le code évite en ne considérant que T tel que $T < S \setminus T$ au sens des masques). En sommant sur toutes les tailles :

$$\sum_{k=0}^n \binom{n}{k} 2^k = (1 + 2)^n = 3^n,$$

d'où la complexité en $O(3^n)$ en temps et en $O(2^n)$ en mémoire (un coût optimal stocké par sous-ensemble). Cette explosion combinatoire – confirmée empiriquement en Table 3, où le temps de calcul est déjà supérieur à 10 secondes à $n = 14$ et dépasse 75 secondes à $n = 16$ – est ce qui limite le DP au rôle de *vérité terrain sur petites instances* uniquement, et justifie l'usage d'une méthode approchée (MCTS) au-delà.

3.2 Greedy agglomératif

Principe. Greedy applique une stratégie *myope* : à chaque étape, il choisit, parmi toutes les paires de groupes actuellement connectées, celle dont le coût de fusion *immédiat* est minimal, sans jamais reconsidérer une décision passée ni anticiper les fusions futures.

Formalisation. Partant de la forêt initiale F_0 des n singletons, Greedy construit une séquence d'états $F_0, F_1, \dots, F_{n-1}$ où, à chaque étape t , la paire fusionnée est

$$(G_i^*, G_j^*) = \underset{\substack{(G_i, G_j) \in F_t \times F_t \\ G_i, G_j \text{ connectés}}}{\text{argmin}} \text{cost}(G_i, G_j),$$

et $F_{t+1} = (F_t \setminus \{G_i^*, G_j^*\}) \cup \{G_i^* \cup G_j^*\}$. Le coût final rapporté est $\sum_i \text{card}(i) + \sum_{t=0}^{n-2} \text{cost}(G_i^*, G_j^*)_t$ (les scans de base plus la somme des coûts de fusion réellement effectués).

Complexité et limite structurelle. À chaque étape, évaluer toutes les paires connectées coûte au pire $O(k^2)$ où $k = |F_t|$ est le nombre de groupes courants, ce qui donne un total en $O(n^3)$ sur l'ensemble du processus – linéaire à sous-cubique, donc bien moins coûteux que le DP exact ou que MCTS. Le prix de cette rapidité est l'absence totale de retour en arrière : un choix localement optimal à l'étape t peut irrémédiablement enfermer l'algorithme dans une structure d'arbre globalement sous-optimale, comme illustré empiriquement en Section 5.6 (étude de cas) où Greedy s'écarte de l'optimum d'un facteur $10\,400\times$ à cause d'un unique mauvais choix à la première étape.

3.3 MCTS

Principe général. MCTS explore l'espace des séquences de fusions en construisant progressivement un arbre de recherche dont chaque nœud correspond à un état (une forêt partiellement construite) et chaque arête à une action (une fusion). Contrairement à Greedy, MCTS ne s'engage pas définitivement sur la fusion qui paraît la meilleure localement : il alloue un budget de N simulations complètes (*itérations*), et à chaque itération, il exécute les quatre phases standard de l'algorithme [4, 5], adaptées ici au cadre *single-player* [6] (un seul agent, pas d'adversaire, minimisation d'un coût plutôt que maximisation d'un taux de victoire).

1. Sélection – UCB1. Partant de la racine (forêt des n singletons), tant que le nœud courant a déjà tous ses enfants créés, on descend l'arbre en choisissant à chaque niveau l'action a qui maximise le score *UCB1* (*Upper Confidence Bound*) :

$$\text{UCB1}(a) = \underbrace{-\bar{\ell}(a)}_{\text{exploitation}} + c \underbrace{\sqrt{\frac{\ln(N_{\text{parent}} + 1)}{n_a}}}_{\text{exploration}}, \quad \bar{\ell}(a) = \frac{1}{n_a} \sum_{k=1}^{n_a} \log(\text{cost}_k),$$

où n_a est le nombre de fois où l'action a (l'enfant correspondant) a été visitée, N_{parent} le nombre de visites du nœud parent, $c = 1.4$ la constante d'exploration, et $\bar{\ell}(a)$ le coût moyen (en *log*) observé après avoir emprunté a . Le terme d'exploitation est **négligé** par rapport au coût moyen car MCTS cherche ici à *minimiser* un coût plutôt qu'à maximiser une récompense : plus $\bar{\ell}(a)$ est petit (branche prometteuse), plus $-\bar{\ell}(a)$ est grand, donc plus l'action est favorisée. Le terme d'exploration, standard dans le cadre des bandits multi-bras [4], garantit que les branches peu visitées restent régulièrement réévaluées malgré un coût moyen provisoirement défavorable. Un enfant jamais visité ($n_a = 0$) reçoit un score infini et est donc toujours exploré en priorité.

2. Expansion. Une fois qu'un nœud possède au moins une action non encore essayée, on en tire une au hasard parmi les actions légales restantes et on crée le nœud enfant correspondant, qui devient le point de départ de la phase de simulation.

3. Simulation (rollout). Depuis le nœud nouvellement créé, on complète la séquence de fusions jusqu'à l'état terminal (un seul groupe restant) par une politique *semi-aléatoire* : à chaque étape du rollout, avec probabilité p_{greedy} (par défaut 0.75) on choisit la fusion la moins chère dans l'immédiat (comportement Greedy), et avec probabilité $1 - p_{\text{greedy}}$ on choisit une fusion légale uniformément au hasard. Formellement, la politique de rollout π_{rollout} tire, à chaque étape, l'action

$$a \sim \begin{cases} \delta\left(\arg\min_{(G_i, G_j)} \text{cost}(G_i, G_j)\right) & \text{avec probabilité } p_{\text{greedy}}, \\ \mathcal{U}(\{\text{actions légales}\}) & \text{avec probabilité } 1 - p_{\text{greedy}}, \end{cases}$$

où $\delta(\cdot)$ désigne la masse de Dirac sur l'action gloutonne et $\mathcal{U}$ la loi uniforme discrète. Ce mélange interpole continûment entre recherche aléatoire pure ($p_{\text{greedy}} = 0$) et une seule trajectoire gloutonne déterministe ($p_{\text{greedy}} = 1$) ; l'effet de ce curseur sur la qualité finale est précisément ce que quantifie l'ablation de la Section 5.5.

4. Rétropropagation. Soit $\text{cost}_{\text{total}}$ le coût complet du plan obtenu à l'issue du rollout (scans de base + toutes les fusions, sélection puis expansion puis simulation). On rétropropage $\ell = \log(\text{cost}_{\text{total}})$ le long de tous les nœuds du chemin parcouru pendant cette itération (de la racine jusqu'au nœud créé en expansion) : pour chaque nœud nd du chemin,

$$\text{visits}(\text{nd}) += 1, \quad \text{total_value}(\text{nd}) += \ell,$$

si bien que $\bar{\ell}(\text{nd}) = \text{total_value}(\text{nd})/\text{visits}(\text{nd})$ utilisé dans la formule UCB1 ci-dessus est bien la moyenne cumulée du log-coût de toutes les simulations passées par ce nœud. Le passage au logarithme (plutôt que le coût brut) est nécessaire car les coûts observés s'étendent sur plusieurs ordres de grandeur (Section 5.1) : sans lui, une poignée de rollouts à coût extrême dominerait la moyenne et rendrait le signal de récompense quasiment inexploitable pour guider la sélection.

Warm-start. MCTS est initialisé avec la solution Greedy comme meilleur coût courant C_{best} avant même la première itération (`seed_with_greedy`), sauf dans l'expérience d'ablation (Section 5.5) où ce warm-start est désactivé afin d'isoler l'effet du seul biais du rollout, sans qu'un plan Greedy initial ne vienne « sauver » artificiellement les configurations peu guidées (p_{greedy} faible).

Convergence asymptotique. Kocsis et Szepesvári [4] montrent que, pour un nombre d'itérations $N \rightarrow \infty$, la probabilité de ne pas avoir identifié l'action optimale à la racine décroît polynomialement en $1/N$ (borne de type *Hoeffding-UCB*), sous réserve que la constante d'exploration c soit correctement calibrée par rapport à l'amplitude des récompenses. En pratique, à budget fini, cette garantie ne se traduit pas nécessairement par une exploration suffisante à grand facteur de branchement : c'est précisément ce qui est mis en évidence empiriquement à $n = 25$ (Section 5.4), où le budget fixe favorise une recherche par redémarrages indépendants plutôt que la gestion d'arbre de MCTS.

3.4 Baselines à budget de calcul équitable

Pour vérifier que l'avantage de MCTS provient réellement de la recherche arborescente guidée et non simplement d'un budget de calcul supérieur à Greedy, nous introduisons deux baselines recevant exactement le même nombre de rollouts complets K que MCTS a d'itérations N (c'est-à-dire $K = N$) : chacune tire K séquences de fusions indépendantes selon la politique π_{rollout} définie ci-dessus, et conserve la meilleure :

$$C_{\text{baseline}} = \min_{k=1,\dots,K} \text{cost}_{\text{total}}^{(k)}, \quad \text{cost}_{\text{total}}^{(k)} \sim \pi_{\text{rollout}}(p_{\text{greedy}}).$$

- **Recherche aléatoire pure** ($p_{\text{greedy}} = 0$ dans le rollout) : K séquences de fusions entièrement aléatoires, on garde la meilleure ;
- **Greedy randomisé avec redémarrages** ($p_{\text{greedy}} = 0.75$, identique au biais par défaut du rollout MCTS) : K séquences partiellement guidées, on garde la meilleure.

La différence essentielle avec MCTS est l'**absence totale de mémoire entre les K tirages** : chaque rollout de ces baselines repart de zéro depuis la racine, sans jamais exploiter l'information accumulée sur les branches déjà explorées, contrairement à MCTS où chaque itération affine l'arbre construit par les itérations précédentes via UCB1. Cette comparaison à budget égal (approximatif : chaque itération MCTS effectue aussi un rollout, plus une gestion d'arbre légère) permet donc de distinguer la contribution propre de la structure d'arbre guidée par UCB1 de celle du simple ré-échantillonnage répété.

4 Protocole expérimental

Toutes les expériences sont reproductibles (graines aléatoires fixées, `seed = 1000n + rep` ou variantes). Le code est structuré en sept expériences :

Expérience	Tailles n	Répétitions	Objectif
(A) Petite échelle	4, 6, 8, 10, 12, 14	10	DP vs Greedy vs MCTS
(B) Grande échelle	16, 20, 25, 30, 35, 40	4	Greedy vs MCTS (sans DP)
(C) Convergence	16, 30, 40	1 (2000 itér.)	Vitesse de convergence de MCTS
(D) Budget équitable	10, 14, 20, 25	4	Isoler l'apport de l'arbre MCTS
(E) Ablation p_{greedy}	20	5×5 valeurs	Effet du biais du rollout
(F) Étude de cas	10 (pire instance)	1	Visualiser un échec de Greedy
(G) Significativité	–	–	IC bootstrap + test de permutation
(H) Progressive Widening	20, 25, 30	8	Tester l'hypothèse de (D) à $n = 25$
(I) Benchmark SHOT/NMCS	10, 14, 20, 25	4 (= instances de D)	L'échec de (D) est-il spécifique à UCB1 ?

TABLE 1 – Vue d'ensemble du protocole expérimental.

Sauf mention contraire, MCTS utilise 1000 itérations à petite et moyenne échelle, et un budget croissant avec n à grande échelle ($\max(1000, 40n)$ itérations) pour rester représentatif d'un usage réel où le budget de recherche est adapté à la taille du problème.

5 Résultats

5.1 Petite échelle : comparaison à l'optimum exact

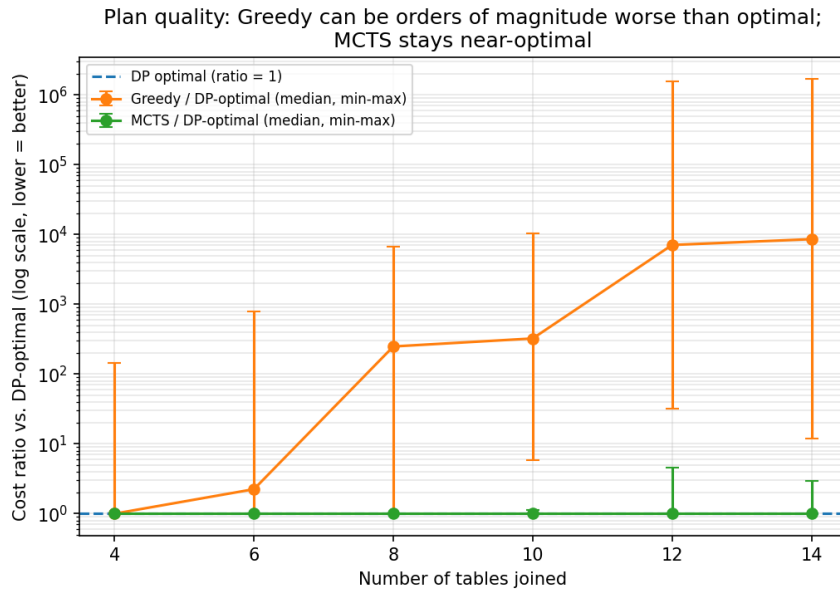
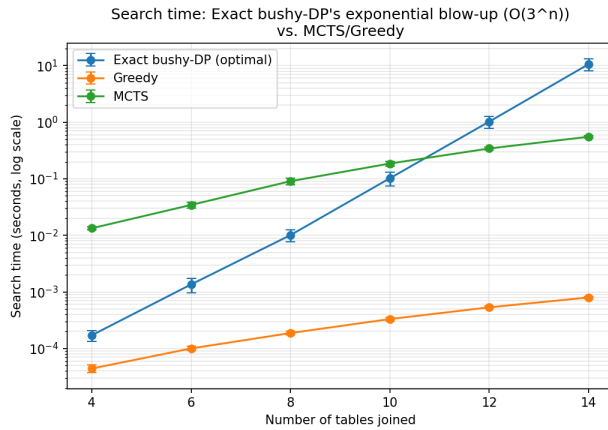


FIGURE 1 – Ratio de coût par rapport à l'optimum DP (médiane, min-max) pour Greedy et MCTS, $n = 4$ à 14, 10 répétitions par taille.

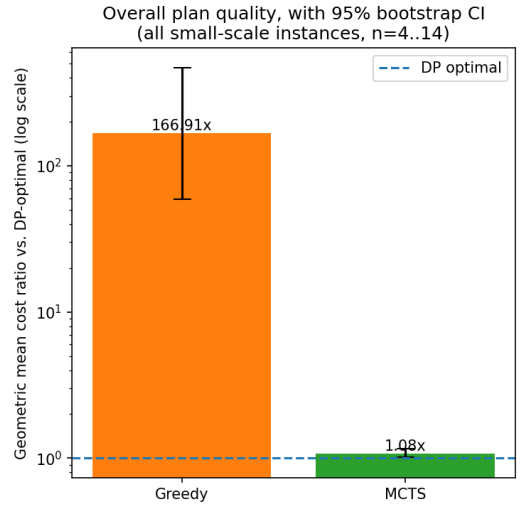
n	Greedy (médiane)	Greedy (max)	MCTS (médiane)	MCTS (max)
4	1.00	142.66	1.0000	1.0000
6	2.24	797.51	1.0000	1.0001
8	248.98	6 784.26	1.0000	1.0002
10	324.01	10 400.61	1.0000	1.1422
12	7 090.25	1 576 964.68	1.0000	4.5120
14	8 585.42	1 685 606.92	1.0000	2.9620

TABLE 2 – Ratios de coût par rapport à l’optimum DP, par taille d’instance (10 répétitions chacune).

Le Tableau 2 montre que Greedy reste parfois optimal (médiane = 1.00 à $n = 4$) mais peut aussi se tromper de plus de six ordres de grandeur dans le pire cas observé ($1\,685\,606\times$ à $n = 14$). MCTS retrouve la médiane exacte de l’optimum à toutes les tailles testées, mais **pas systématiquement** : à $n = 14$, deux des dix répétitions ratent l’optimum d’un facteur 1.59 et 2.96 respectivement, signe que le budget fixe de 1000 itérations commence à ne plus suffire à couvrir exhaustivement l’espace de recherche à cette taille.



(a) Temps de recherche (échelle log) : le DP exact explose en $O(3^n)$ tandis que Greedy et MCTS restent sous-exponentiels.



(b) Moyenne géométrique du ratio de coût vs. optimum DP, avec intervalle de confiance bootstrap à 95 %, sur les 60 instances petite échelle.

n	Temps DP moyen (s)
4	0.0002
6	0.0014
8	0.0101
10	0.1022
12	1.0246
14	10.6174

TABLE 3 – Temps d’exécution moyen du DP exact, croissance conforme à $O(3^n)$. À $n = 16$ (mesuré séparément, hors protocole principal), le DP dépasse 75 secondes – déjà impraticable.

En résumé sur les 60 instances de petite échelle : Greedy affiche une moyenne géométrique de $166.91\times$ l’optimum (IC bootstrap 95 % : $[58.90, 469.20]$), contre $1.08\times$ pour MCTS (IC 95 % : $[1.02, 1.16]$). Un test de permutation apparié (« sign-flip », 20 000 permutations) sur

$\log(\text{ratio}_{\text{Greedy}}) - \log(\text{ratio}_{\text{MCTS}})$ par instance rejette l'hypothèse nulle d'absence de différence systématique avec $p < 5 \times 10^{-5}$ (résolution du test limitée par le nombre de permutations).

5.2 Grande échelle

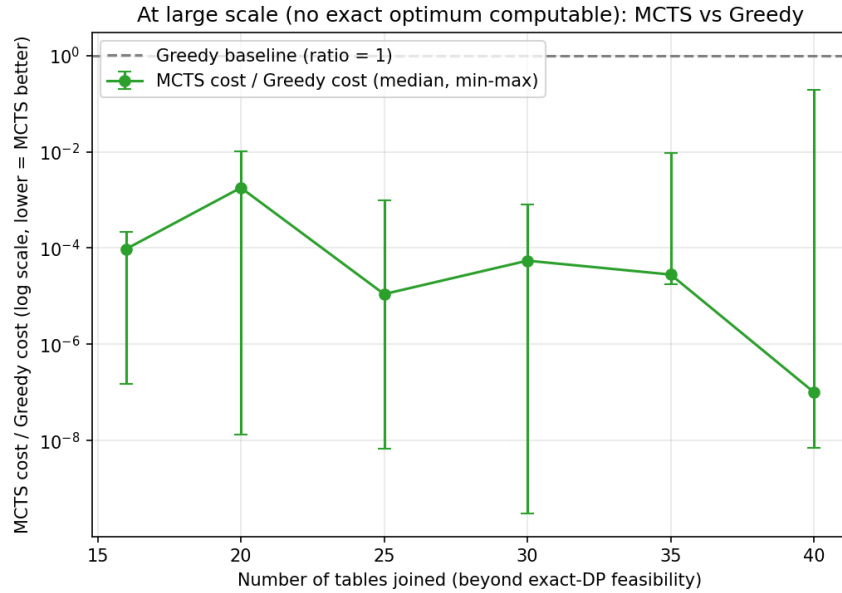


FIGURE 3 – Ratio de coût MCTS/Greedy (médiane, min-max) pour $n = 16$ à 40 , sans optimum DP disponible.

Sans référence DP disponible au-delà de $n = 14$, nous comparons directement MCTS à Greedy. Le ratio médian MCTS/Greedy reste systématiquement très inférieur à 1 (de l'ordre de 10^{-4} à 10^{-5} selon n), confirmant que l'avantage de MCTS observé à petite échelle se maintient, voire s'amplifie, quand le DP exact n'est plus une option.

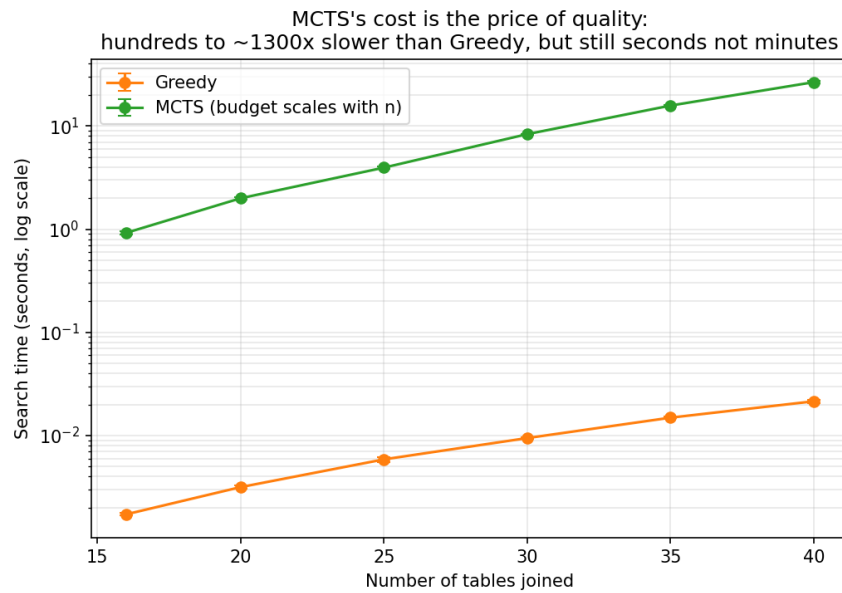


FIGURE 4 – Temps de recherche : le coût de MCTS est le prix de sa qualité de solution (des centaines à $\sim 1300\times$ plus lent que Greedy), mais reste de l'ordre de la seconde à la dizaine de secondes.

Ce gain de qualité n'est pas gratuit : à $n = 40$, MCTS prend environ 27 secondes contre 0.02 seconde pour Greedy, soit un facteur ~ 1300 . Ce compromis temps/qualité doit être explicitement mis en balance dans toute conclusion : MCTS n'est pas « gratuitement meilleur », il dépense délibérément plus de calcul pour cela (cf. Section 3.4 et 7).

5.3 Convergence

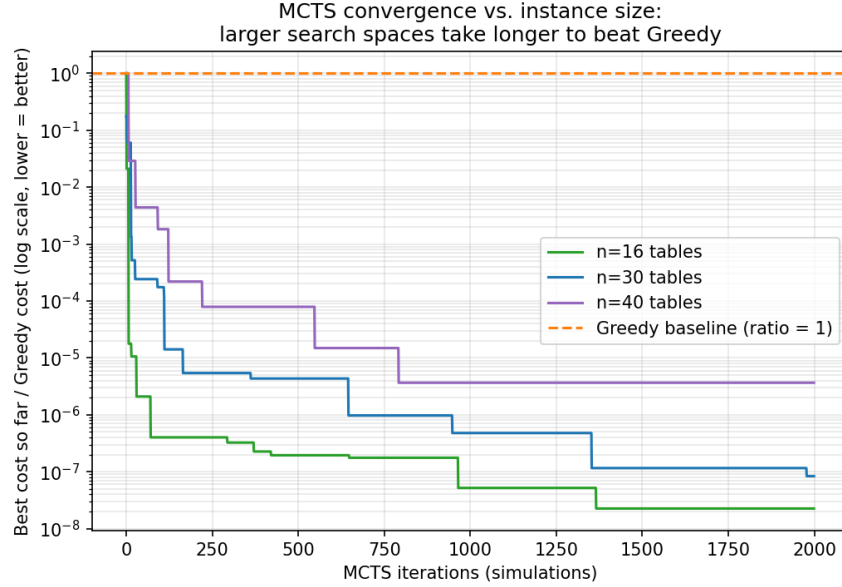


FIGURE 5 – Meilleur coût trouvé au fil des itérations MCTS, normalisé par le coût Greedy, pour trois tailles d'instance.

Les instances plus grandes nécessitent davantage d'itérations pour dépasser Greedy et atteindre leur plateau : à $n = 16$, MCTS bat déjà Greedy dès les toutes premières itérations et se stabilise vers 200–400 itérations, tandis qu'à $n = 40$ la courbe continue de s'améliorer significativement au-delà de 1000 itérations. Ce résultat qualitatif est cohérent avec l'intuition que l'espace de recherche croît avec n et qu'un budget fixe devient relativement plus contraignant.

5.4 Budget de calcul équitable : d'où vient l'avantage de MCTS ?

n	Greedy	Recherche aléatoire	Greedy randomisé + redémarrages	MCTS
10	1 131.75	1.00	1.00	1.00
14	431 809.46	1.46	1.00	1.00
20	9 225 476.91	21.64	2.50	1.00
25	637 165.68	589.68	1.00	5.999

TABLE 4 – Regret médian (coût / meilleur coût trouvé pour l'instance) à budget de rollouts identique pour les quatre méthodes.

C'est le résultat le plus important pour la rigueur de ce travail. Jusqu'à $n = 20$, MCTS retrouve systématiquement (regret médian = 1.00) le meilleur coût parmi les quatre méthodes à budget de rollouts égal, confirmant que son avantage provient bien de la recherche arborescente guidée par UCB1, et non simplement d'un budget de calcul supérieur à Greedy. **Mais à $n = 25$, c'est le « Greedy randomisé avec redémarrages » qui obtient le meilleur coût médian (regret = 1.00), tandis que MCTS a un regret médian de 5.999 et perd face à cette**

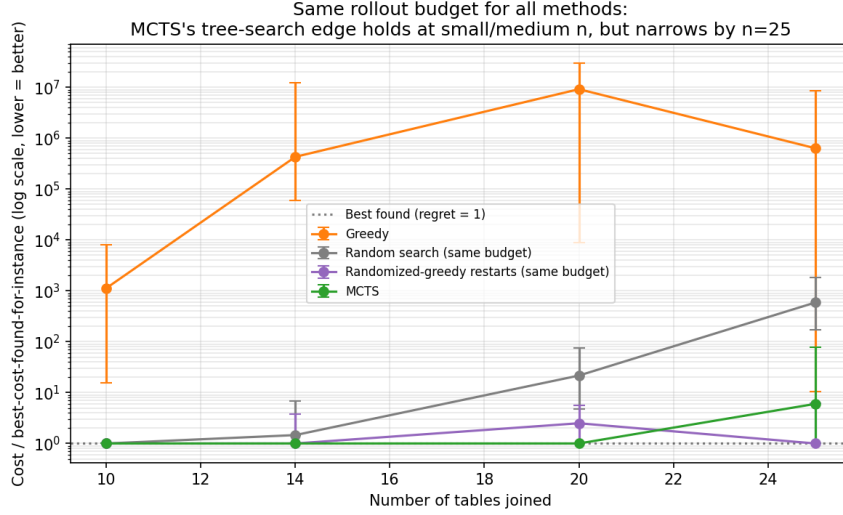


FIGURE 6 – Regret (coût / meilleur coût trouvé parmi les quatre méthodes) à budget de rollouts égal. MCTS domine jusqu’à $n = 20$ mais son avantage s’amenuise nettement à $n = 25$.

baseline dans 3 des 4 répétitions (jusqu’à $78\times$ pire que le meilleur trouvé dans le pire cas individuel). Une explication plausible est que la gestion de l’arbre MCTS (exploration en largeur au niveau de la racine avant d’approfondir) consomme une part croissante du budget fixe à mesure que le facteur de branchement augmente avec n , alors qu’une recherche par redémarrages complets indépendants reste, elle, insensible à ce coût structurel. Ce résultat n’a pas été anticipé et mériterait une investigation complémentaire (le déficit se referme-t-il avec davantage d’itérations à $n = 25$, ou s’agit-il d’une limite structurelle de l’algorithme UCB1 tel que paramétré ici?).

5.5 Ablation du biais du rollout (p_{greedy})

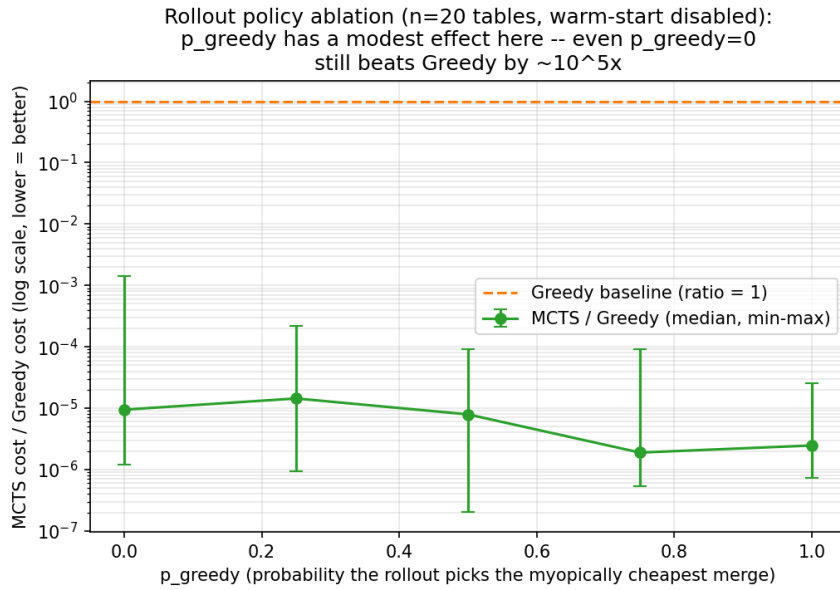


FIGURE 7 – Effet du paramètre p_{greedy} (probabilité que le rollout choisisse la fusion la moins chère plutôt qu’une fusion aléatoire) sur la qualité finale, $n = 20$, warm-start désactivé.

L’hypothèse initiale (non testée) du projet affirmait qu’« un rollout 100 % aléatoire s’effondre

à grande échelle ». Les données mesurées à $n = 20$ contredisent la force de cette affirmation : même avec $p_{\text{greedy}} = 0$ (rollout entièrement aléatoire), MCTS reste environ 10^5 fois moins coûteux que Greedy (ratio médian 9.45×10^{-6}). On observe bien une tendance – le ratio médian diminue globalement de $p_{\text{greedy}} = 0$ à $p_{\text{greedy}} = 0.75$ (de 9.45×10^{-6} à 1.89×10^{-6}) – mais l’effet est modeste et non un effondrement catastrophique à cette taille d’instance. Il est possible qu’un effondrement plus net apparaisse à plus grande échelle ($n > 30$) ou avec un budget d’itérations plus contraint ; cela reste à vérifier.

5.6 Étude de cas qualitative

Sur l’instance de petite échelle présentant le pire ratio Greedy/optimal parmi celles avec $n \leq 10$ ($n = 10$, graine 10005, ratio = 10 400.61 $\times$), les plans obtenus sont :

- **DP optimal** (1.5325×10^{20}) :
 $((((9 \text{ JOIN } 6) \text{ JOIN } 2) \text{ JOIN } (3 \text{ JOIN } 0)) \text{ JOIN } ((8 \text{ JOIN } 4) \text{ JOIN } 5) \text{ JOIN } (7 \text{ JOIN } 1)))$
- **Greedy** (1.5939×10^{24} , 10 400.61 $\times$ l’optimum) :
 $((((3 \text{ JOIN } 7) \text{ JOIN } (1 \text{ JOIN } 6)) \text{ JOIN } (0 \text{ JOIN } 5)) \text{ JOIN } ((2 \text{ JOIN } 9) \text{ JOIN } (8 \text{ JOIN } 4)))$
- **MCTS** (1.5325×10^{20} , 1.0000 $\times$ l’optimum) :
 $((((1 \text{ JOIN } 5) \text{ JOIN } 7) \text{ JOIN } (4 \text{ JOIN } 8)) \text{ JOIN } ((2 \text{ JOIN } (9 \text{ JOIN } 6)) \text{ JOIN } (3 \text{ JOIN } 0)))$

Greedy choisit dès la première étape une paire localement bon marché (impliquant les tables 3 et 7) qui l’engage dans une structure d’arbre globalement sous-optimale, sans possibilité de retour en arrière – l’illustration directe de sa nature myope. MCTS retrouve un plan de coût strictement identique à l’optimum DP (bien que sous une forme d’arbre différente, les deux étant équivalents en coût car la fonction objectif est invariante par permutation au sein d’un même partitionnement). Le détail complet des cardinalités et sélectivités de cette instance est disponible dans `outputs/case_study.txt`.

6 Benchmark MCTS : trois méthodes complémentaires vues en cours

Les sections précédentes comparent MCTS à Greedy et à deux baselines à budget de calcul équitable (recherche aléatoire, Greedy randomisé avec redémarrages – Section 3.4). Le cours a couvert plusieurs autres façons de structurer une recherche à budget fixe pour ce type de problème de construction séquentielle single-agent ; cette section les met à l’épreuve, **exactement sur les mêmes instances** que celles de la Section 5.4 (mêmes graines aléatoires `seed = 3000n + rep`), afin de rester directement comparable au Tableau 4 plutôt que de constituer une expérience isolée sur des instances différentes. Trois méthodes sont ajoutées :

- **Progressive Widening** (PW) – une modification de l’arbre MCTS lui-même, qui teste *directement* l’hypothèse formulée en Section 5.4 pour expliquer l’échec de MCTS à $n = 25$;
- **Sequential Halving appliqué aux arbres** (SHOT) – une troisième façon d’allouer ce même budget fixe entre candidats, sans arbre guidé par UCB1 ;
- **Nested Monte Carlo Search** (NMCS) – une alternative sans aucun arbre, conçue pour la même famille de problèmes de construction séquentielle single-agent (le voyageur de commerce en est l’exemple classique).

Ensemble, elles permettent de répondre à une question laissée ouverte en Section 5.4 : le déficit de MCTS à $n = 25$ face à un simple Greedy randomisé avec redémarrages est-il un défaut *spécifique* à la recherche arborescente guidée par UCB1, ou une limite plus générale de toute méthode qui dépense une partie de son budget fixe à *structurer* la recherche plutôt qu’à seulement l’exécuter en aveugle ?

6.1 Rappel : ce à quoi ces méthodes sont comparées

Par souci de clarté, on rappelle les quatre méthodes déjà en présence (détaillées en Section 3 et 3.4), toutes exécutées avec le même budget total $K = N = 1000$ (le même « nombre de rollouts complets équivalents ») :

- **Greedy** : agglomératif, myope, sans budget de calcul supplémentaire (Section 3.2) – la référence « sans recherche » ;
- **Recherche aléatoire** : K séquences de fusions entièrement aléatoires, on garde la meilleure ;
- **Greedy randomisé avec redémarrages** : K séquences partiellement guidées ($p_{\text{greedy}} = 0.75$), on garde la meilleure – c’est la baseline qui bat MCTS à $n = 25$ dans le Tableau 4 ;
- **MCTS (vanilla)** : arbre guidé par UCB1 (Section 3.4).

La métrique reste le *regret* : coût obtenu divisé par le meilleur coût trouvé, toutes méthodes confondues, sur cette instance précise – exactement comme au Tableau 4 – ce qui permet de comparer sept méthodes sur un pied d’égalité même sans optimum DP disponible ($n > 14$).

6.2 Progressive Widening : cibler directement l’échec à $n = 25$

Principe. MCTS « vanilla » (Section 3.4) expanse *tous* les enfants légaux d’un nœud avant de jamais redescendre dans l’arbre via UCB1 : la phase de sélection ne s’arrête d’explorer en largeur que lorsque plus aucune action légale n’est disponible. Or le nombre d’actions légales à la racine (le nombre de paires de groupes connectées, parmi les n singletons initiaux) croît avec le facteur de branchement, donc avec n – c’est précisément le mécanisme suspecté en Section 5.4 pour expliquer pourquoi un budget fixe devient insuffisant : une part croissante des 1000 itérations est « gaspillée » à simplement créer tous les enfants de la racine, plutôt qu’à approfondir les branches déjà prometteuses. *Progressive Widening* [9] corrige ce comportement en plafonnant le nombre d’enfants qu’un nœud a le droit d’exposer, en fonction de son propre nombre de visites N_{nd} :

$$k(N_{\text{nd}}) = \lceil C \cdot (N_{\text{nd}} + 1)^\alpha \rceil,$$

où C et $\alpha \in (0, 1)$ sont deux hyperparamètres. Un nœud ne peut créer un $(k+1)$ -ième enfant que si $k < k(N_{\text{nd}})$ et qu’il lui reste des actions non essayées ; sinon, la sélection UCB1 s’applique dès à présent parmi les enfants déjà créés, même si d’autres actions légales restent inexplorées. Avec $\alpha = 0.5$ par exemple, un nœud visité 100 fois ne peut exposer que $\sim 10 \times \sqrt{C}$ enfants, quel que soit son facteur de branchement réel – l’arbre est donc contraint à aller en profondeur bien plus tôt qu’en MCTS vanilla, au prix de laisser certaines actions légales définitivement inexplorées à ce nœud si le budget s’épuise avant que le seuil ne les autorise.

Choix des hyperparamètres. C et α ont été réglés au préalable par un petit essai exploratoire indépendant (6 combinaisons de $C \in \{0.5, 1, 2, 3, 4\}$ et $\alpha \in \{0.3, 0.4, 0.5\}$, sur un premier lot de 8 instances $n = 25$ tirées avec les mêmes graines) : $C = 3.0$, $\alpha = 0.4$ y obtient la meilleure moyenne géométrique de regret parmi les combinaisons testées – pas la meilleure sur chaque instance individuelle, mais la plus régulière dans l’ensemble. Cette configuration est ensuite gelée (aucun nouveau réglage) pour l’expérience rapportée ci-dessous, aux trois tailles $n = 20, 25, 30$; les chiffres qui suivent proviennent tous de cette expérience-là, pas de l’essai exploratoire préalable.

Protocole. 8 répétitions à $n = 20, 25$ et 30 (contre 4 dans l’expérience originale de la Section 5.4, pour resserrer l’estimation autour du phénomène qu’on cherche justement à expliquer), avec le même schéma de graines $\text{seed} = 3000n + \text{rep}$: à $n = 20$ et 25 , les répétitions 0 à 3 sont donc les instances exactes du Tableau 4, complétées ici par 4 répétitions supplémentaires (4 à 7) ; $n = 30$ étend l’exploration au-delà de la plage déjà testée en Section 5.4.

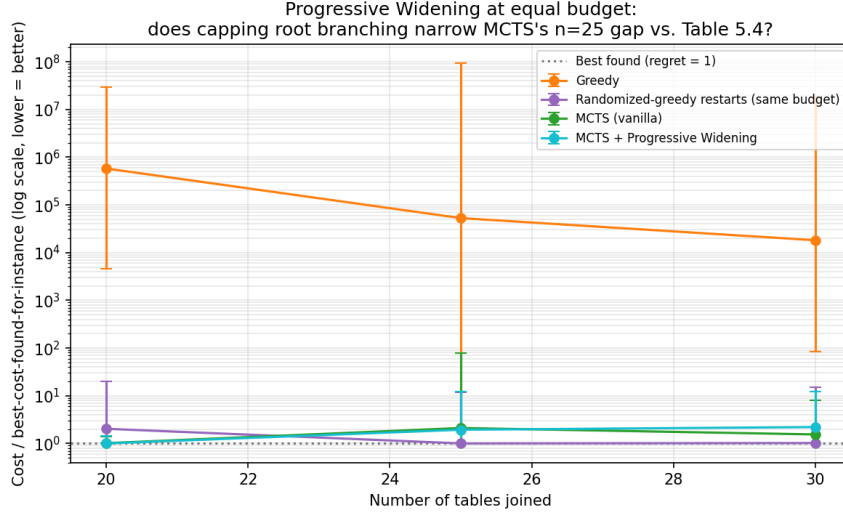


FIGURE 8 – Regret (coût / meilleur coût trouvé parmi les quatre méthodes) à budget de rollouts égal, avec et sans Progressive Widening, $n = 20, 25, 30$ (médiane, min-max, 8 répétitions).

n	Greedy randomisé + redémarrages	MCTS (vanilla)	MCTS + Progressive Widening
20	2.03	1.02	1.00
25	1.00	2.11	1.94
30	1.02	1.55	2.21

TABLE 5 – Regret médian (8 répétitions) à budget de rollouts identique. Greedy (sans budget de recherche) est omis de ce tableau – son regret médian reste de plusieurs ordres de grandeur (voir Figure 8).

Résultat : une confirmation partielle, pas une solution complète. À $n = 25$, exactement l’instance qui a motivé cette expérience, Progressive Widening améliore MCTS : la moyenne géométrique du regret passe de $3.11\times$ (vanilla) à $2.17\times$ (avec PW) sur les 8 répétitions, et la médiane de $2.11\times$ à $1.94\times$ – un gain réel, qui rapproche MCTS de la baseline Greedy randomisé + redémarrages (1.00 – $2.06\times$ selon la statistique) sans toutefois l’égaliser ni la dépasser. **Mais ce gain ne se généralise pas proprement à $n = 30$** : sur cet échantillon, PW dégrade légèrement MCTS (regret médian $1.55\times \rightarrow 2.21\times$). L’hypothèse de la Section 5.4 (le gaspillage de budget en largeur à la racine) est donc *partiellement* confirmée – limiter la largeur aide bien dans le régime précis où le problème a été identifié – mais Progressive Widening telle que réglée ici n’est pas une solution générale à la dégradation de MCTS à budget fixe : elle pourrait nécessiter un réglage de (C, α) dépendant de n , ce qui n’a pas été exploré plus avant faute de temps.

6.3 Sequential Halving appliqué aux arbres (SHOT)

Principe. *Sequential Halving* [10] est un algorithme d’identification du meilleur bras à budget fixe : au lieu d’échantillonner tous les bras uniformément (comme la recherche aléatoire) ou de les pondérer par un score d’exploration-exploitation (comme UCB1), Sequential Halving procède par *tours* – à chaque tour, chaque bras survivant reçoit le même petit nombre d’essais supplémentaires, puis la moitié la moins bonne (au sens du coût moyen observé) est éliminée, jusqu’à n’en garder qu’un seul. *SHOT* [11] applique cette même idée comme politique arborescente, à *chaque point de décision* du plan de jointure (et pas seulement une fois à la racine) : pour choisir la fusion à effectuer, les candidats légaux sont départagés par Sequential Halving, chaque candidat étant évalué par le coût moyen (en *log*, pour la même raison que la rétropropagation de MCTS – Section 3.4 – les coûts s’étendant sur plusieurs ordres de grandeur) d’un rollout complet joué

après lui. Le budget total $K = 1000$ est ici réparti *également* entre les $n - 1$ décisions de la séquence de fusions – un choix de conception simple, qui défavorise mécaniquement la première décision (celle qui a historiquement le plus grand nombre de candidats, et dont la Section 3.2 montre qu’elle est aussi la plus critique) par rapport aux décisions tardives, moins nombreuses mais tout aussi bien dotées en budget.

Résultats. La Figure 9 et le Tableau 6 montrent que SHOT bat toujours Greedy largement, mais son avantage relatif s’effondre à mesure que n croît : en moyenne géométrique, SHOT réduit le regret de Greedy d’un facteur $\sim 26\times$ à $n = 10$ ($392.6 \rightarrow 15.2$), puis d’un facteur $\sim 1350\times$ à $n = 14$, mais à $n = 25$ SHOT ($32\,345.7\times$) n’est plus significativement meilleur que Greedy lui-même ($31\,078.9\times$) – l’écart entre les deux moyennes géométriques est négligeable au vu de la variance. La répartition uniforme du budget entre décisions, adaptée pour des séquences courtes, devient intenable quand le nombre de candidats à la première décision explose avec n : chaque candidat de la racine ne reçoit plus qu’une poignée d’essais, rendant le tri par Sequential Halving à peine plus informatif qu’un choix aléatoire à cette étape précise – alors que cette première décision est, comme démontré par l’étude de cas (Section 5.6), celle qui verrouille le plus la qualité finale.

6.4 Nested Monte Carlo Search (NMCS)

Principe. *NMCS* [12] construit la séquence de fusions sans jamais bâtir d’arbre persistant. Au niveau 0, c’est un simple rollout. Au niveau $L > 0$, à chaque étape de la construction, NMCS essaie *toutes* les fusions légales, évalue chacune en appelant NMCS au niveau $L - 1$ depuis l’état résultant, et s’engage sur celle qui obtient le meilleur score – tout en retenant, en mémoire, la meilleure séquence complète jamais rencontrée n’importe où dans la récursion (un appel de niveau $L - 1$ profondément imbriqué dans une branche par ailleurs médiocre peut malgré tout tomber sur un excellent plan complet, que la marche gloutonne de plus haut niveau n’aurait pas choisi). Le niveau 1 est utilisé ici : le niveau 2 imbriquerait un facteur supplémentaire du facteur de branchement à chaque étape (coût total en $O(n^4)$ contre $O(n^3)$ pour le niveau 1), ce qui devient rapidement intraitable au facteur de branchement de nos graphes de jointure denses ($p_{\text{edge}} = 0.4$).

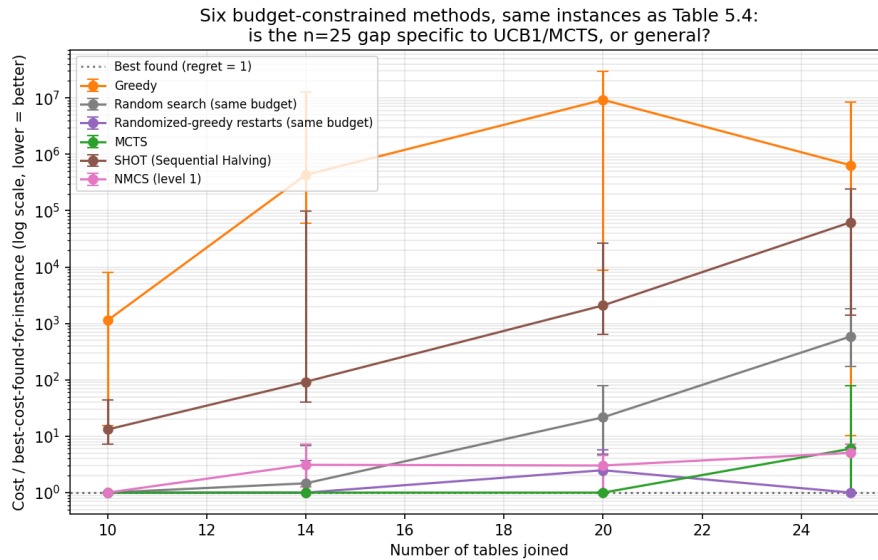


FIGURE 9 – Six méthodes à budget de rollouts égal, sur les mêmes instances que le Tableau 4 : SHOT s’effondre avec n , NMCS se dégrade plus modérément, MCTS reste la plus robuste jusqu’à $n = 20$.

n	Rech. aléatoire	Greedy rand. + redém.	MCTS	SHOT	NMCS (niv. 1)
10	1.00	1.00	1.00	13.16	1.00
14	1.46	1.00	1.00	92.32	3.11
20	21.64	2.50	1.00	2 086.10	3.02
25	589.68	1.00	6.00	62 375.62	5.07

TABLE 6 – Regret médian (4 répétitions, mêmes instances que le Tableau 4) à budget de rollouts identique pour six méthodes (Greedy, omis ici, reste à plusieurs ordres de grandeur – voir Figure 9).

Résultats. NMCS bat systématiquement Greedy et SHOT à toutes les tailles testées, avec une dégradation nettement plus douce que celle de SHOT à mesure que n croît (regret géométrique moyen $1.00 \rightarrow 2.95 \rightarrow 2.48 \rightarrow 5.51$ de $n = 10$ à 25 , contre $15.2 \rightarrow 427.5 \rightarrow 2792.7 \rightarrow 32\,345.7$ pour SHOT) – remarquable pour une méthode qui, contrairement à MCTS et SHOT, ne maintient aucune mémoire d’un nœud à l’autre au sein d’une même recherche. NMCS reste cependant en retrait par rapport à MCTS jusqu’à $n = 20$, et par rapport à Greedy randomisé + redémarrages à $n = 25$ ($5.07\times$ contre $1.00\times$ de regret médian) : sans arbre ni bandit pour orienter la recherche, chaque étape de sa construction gloutonne coûte un rollout complet par candidat, ce qui consomme vite le budget disponible à mesure que le nombre de décisions et de candidats par décision augmentent avec n .

6.5 Synthèse du benchmark

Revenons à la question posée en introduction de cette section : l’échec de MCTS à $n = 25$ (Section 5.4) est-il spécifique à UCB1, ou général à toute méthode qui structure sa recherche plutôt que de dépenser tout son budget en rollouts complets indépendants ? Les résultats penchent vers la seconde réponse, **avec des nuances importantes** :

- **SHOT (tree search sans UCB1) dégrade bien plus sévèrement que MCTS** – à $n = 25$, SHOT perd tout avantage mesurable sur Greedy, là où MCTS conserve un avantage net ($6.00\times$ de regret médian, très loin des $637\,165.68\times$ de Greedy). Le point commun entre SHOT et MCTS – répartir un budget fixe entre de nombreuses décisions séquentielles plutôt que sur des rollouts complets indépendants – semble donc bien être un facteur de fragilité partagé, pas un artefact de UCB1 en particulier.
- **NMCS (sans arbre du tout) se dégrade aussi, mais plus modérément que SHOT** – elle reste nettement au-dessus de Greedy et de SHOT à toutes les tailles testées, sans pour autant égaler MCTS ou Greedy randomisé + redémarrages à $n = 25$.
- **Progressive Widening apporte un gain réel mais partiel**, spécifiquement dans le régime ($n = 25$) où le problème a été identifié, sans se généraliser proprement à $n = 30$ avec les mêmes hyperparamètres.

Ce panorama élargi (sept méthodes en tout, en comptant Greedy) renforce la conclusion positive des sections précédentes – MCTS reste, de loin, la méthode la plus robuste parmi celles testées jusqu’à $n = 20$ – tout en montrant que le régime de budget contraint à grande échelle ($n \geq 25$) reste une difficulté ouverte et partagée par plusieurs familles de méthodes de recherche, pas seulement par l’implémentation particulière de MCTS testée ici.

7 Discussion et limites

Ce que ces résultats supportent. Sur des instances de taille petite à moyenne ($n \leq 20$), et à condition de disposer d’un budget de calcul suffisant, MCTS retrouve quasi systématiquement l’ordre de jointure optimal ou quasi-optimal, là où Greedy peut se tromper de plusieurs ordres de grandeur. L’écart est statistiquement significatif et la source de l’avantage (la recherche arbo-

rescente elle-même, pas seulement le budget de calcul disponible) est confirmée par l’expérience à budget égal.

Ce qu’ils ne supportent pas. L’affirmation « MCTS bat toujours Greedy » n’est pas soutenue sans nuance : à $n = 25$ et à budget fixe, une baseline plus simple (Greedy randomisé avec redémarrages) devient compétitive, voire supérieure. De même, la nécessité d’un rollout biaisé vers Greedy pour éviter un « effondrement » à grande échelle n’est que partiellement confirmée à $n = 20$ – l’effet mesuré est réel mais modeste. Le benchmark de la Section 6 montre par ailleurs que ce déficit à $n = 25$ n’est pas propre à UCB1 (SHOT, une autre méthode arborescente à budget fixe, s’effondre bien davantage), mais aussi que Progressive Widening – le correctif le plus directement ciblé sur l’hypothèse avancée en Section 5.4 – n’apporte qu’une amélioration partielle à $n = 25$ et ne se généralise pas proprement à $n = 30$.

Limites méthodologiques.

- **Taille d’échantillon** : 10 répétitions (petite échelle) et 4 répétitions (grande échelle, budget équitable) par configuration restent modestes au vu de la variance considérable observée entre instances (les ratios s’étendent sur plus de six ordres de grandeur). Des intervalles de confiance plus étroits nécessiteraient davantage de répétitions.
- **Générateur d’instances** : les graphes de jointure sont générés avec des paramètres délibérément choisis pour créer des pièges pour Greedy (densité 0.4, sélectivités très variées). Cela démontre l’existence du phénomène mais ne garantit pas sa représentativité sur des graphes de jointure réels (schémas en étoile, en flocon, etc.).
- **Budget MCTS non strictement égal en temps CPU** : les baselines à budget équitable (Section 5.4) reçoivent le même nombre de rollouts complets que MCTS a d’itérations, ce qui est une approximation raisonnable mais pas une égalité stricte (MCTS effectue aussi une gestion d’arbre en plus du rollout par itération).
- **Absence de benchmark réel** : aucune validation sur un jeu de requêtes réelles (par exemple le *Join Order Benchmark* [7], basé sur IMDB) n’a été effectuée ; les cardinalités et sélectivités restent synthétiques.

Pistes futures. Le benchmark de la Section 6 répond partiellement à l’une des pistes envisagées initialement (Progressive Widening comme correctif à l’hypothèse de la Section 5.4), mais son échec à se généraliser à $n = 30$ suggère qu’un réglage de (C, α) dépendant de n , ou une combinaison avec du partage de statistiques entre branches (AMAF/RAVE, non testé ici), mériterait une investigation dédiée. Reproduire l’expérience à budget équitable (Section 5.4) à $n > 30$ avec un budget croissant permettrait aussi de déterminer si l’écart se referme ou se creuse davantage. Une étude de sensibilité à la densité et à la topologie du graphe de jointure (chaîne, étoile, cyclique) relierait plus directement les résultats à la distinction left-deep/bushy et IKKBZ évoquée en introduction. Enfin, une validation sur le Join Order Benchmark donnerait une portée pratique directement comparable à la littérature en bases de données.

8 Conclusion

Ce travail montre que MCTS constitue une méthode de recherche pertinente et empiriquement efficace pour l’optimisation de l’ordre de jointures bushy, un problème NP-difficile où l’heuristique gloutonne classique peut échouer de façon spectaculaire faute d’anticipation. Au-delà de la démonstration attendue, ce rapport a cherché à isoler précisément la source de cet avantage (recherche arborescente vs simple budget de calcul) et à documenter honnêtement les régimes où cet avantage s’amenuise. Le benchmark élargi de la Section 6 – Progressive Widening, Sequential Halving appliqué aux arbres, Nested Monte Carlo Search – confirme que cette dégradation à grande échelle et budget fixe n’est pas une bizarrerie propre à UCB1, mais une difficulté partagée,

à des degrés divers, par plusieurs familles de méthodes de recherche vues en cours ; parmi elles, MCTS reste la plus robuste jusqu'à $n = 20$, et Progressive Widening n'offre qu'un remède partiel au-delà. Cette approche – mesurer plutôt que supposer, et rapporter les résultats qui nuancent la thèse principale plutôt que de les omettre – nous semble être ce qui distingue une évaluation empirique rigoureuse d'une simple démonstration de faisabilité.

Reproductibilité

L'ensemble du code (générateur d'instances, modèle de coût, algorithmes, expériences, figures) est organisé comme suit :

```
join_graph.py # generateur d'instances synthetiques
cost_model.py # modele de cout (hash-join)
bushy_algorithms.py # DP exact, Greedy, MCTS (+ Progressive Widening),
                    # baselines a budget equitable, SHOT, NMCS
stats_utils.py # moyenne geometrique, IC bootstrap, test de permutation
experiments.py # les 9 experiences (A-I)
plots.py # generation des 11 figures
outputs/ # resultats (CSV, TXT) et figures (PNG)
```

Toutes les expériences sont ré-exécutables via `python experiments.py` suivi de `python plots.py`, avec des graines aléatoires fixées pour la reproductibilité intégrale des résultats rapportés ici.

Références

- [1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, 1979.
- [2] T. Ibaraki and T. Kameda. On the optimal nesting order for computing N-relational joins. *ACM Transactions on Database Systems*, 9(3), 1984.
- [3] R. Krishnamurthy, H. Boral, and C. Zaniolo. Optimization of nonrecursive queries. In *Proceedings of the 12th International Conference on Very Large Data Bases (VLDB)*, 1986.
- [4] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning (ECML)*, 2006.
- [5] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavenner, D. Perez, S. Samothrakis, and S. Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 2012.
- [6] M. P. D. Schadd, M. H. M. Winands, H. J. Van Den Herik, G. M. J.-B. Chaslot, and J. W. H. M. Uiterwijk. Single-player Monte-Carlo tree search. In *International Conference on Computers and Games*, 2008.
- [7] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. How good are query optimizers, really? *Proceedings of the VLDB Endowment*, 9(3), 2015.
- [8] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul. Neo : A learned query optimizer. *Proceedings of the VLDB Endowment*, 12(11), 2019.
- [9] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. Van Den Herik, J. W. H. M. Uiterwijk, and B. Bouzy. Progressive strategies for Monte-Carlo tree search. *New Mathematics and Natural Computation*, 4(3), 2008.
- [10] Z. Karnin, T. Koren, and O. Somekh. Almost optimal exploration in multi-armed bandits. In *International Conference on Machine Learning (ICML)*, 2013.

- [11] T. Cazenave. Sequential halving applied to trees. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(1), 2014.
- [12] T. Cazenave. Nested Monte-Carlo search. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2009.